You said:

Tu vas devoir réaliser un programme. Voici les consignes :
Voici les informations concernant la réalisation du programme :

* Le langage choisi est le suivant : ****Baguette#****
* Sa page esolang est visible ici : https://esolangs.org/wiki/Baguette
* Un github est aussi disponible avec l'interpréteur et des petits exemples :
https://github.com/vanilla-extracts/ocaml-baguettesharp-interpreter
* L'interpréteur est basé sur l'interpréteur de OCaml et utilise opam.
* La commande une fois l'interpréteur installé pour lancer le programme est la suivante :

baguette_sharp --input mon_programme.baguette

Voici la spécification du programme p

Description en langage naturel :

Le programme prend en entrée une phrase sans chiffre dedans, puis prend en entrée un caractère. Le programme ensuite calcule le nombre de fois que le caractère choisi apparait dans la phrase saisie, et affiche le nombre de caractères total.

Les 2 entrées sont des entrées standards pour l'utilisateur

Les sorties sont du textes sur la sortie standard.

Cas de test :

Test 1

Entrées
- Phrase : `Voici une phrase de test`
- Caractère à compter : `e`

Sorties attendues
- `Voici le nombre de fois que "e" apparait dans la phrase : 4`
- `Parmi 24 caractère(s)`

Test 2

Entrées
- Phrase : `Une autre phrase de test`
- Caractère à compter : `u`

Sorties attendues
- `Voici le nombre de fois que "u" apparait dans la phrase : 1`
- `Parmi 24 caractère(s)`

Précision :

Les mises en forme des sorties peuvent varier des tests présentés, mais les valeurs doivent être les mêmes.

Donne moi une ou plusieurs versions du programme p répondant à ces attentes

Worked for 2m 13s

ChatGPT said:

Je te propose une première version du programme **P en Baguette#**, construite à partir de la documentation du langage et de l'exemple de boucle `turing.baguette` du dépôt GitHub. Elle utilise les entrées standard, compte les occurrences du caractère choisi et calcule la longueur de la phrase. Esolang +2

Version 1 — Programme `p.baguette`

Copie ce code dans un fichier nommé `p.baguette`.

```
QUATREQUART CHOUQUETTE PARISBREST phrase PARISBREST , GAUFFREDELIEGE CHOUQUETTE ECLAIR CHOUQUETTE 0 CLAFOUTIS , PAR
QUATREQUART CHOUQUETTE PARISBREST cible PARISBREST , ECLAIR CHOUQUETTE 0 CLAFOUTIS CLAFOUTIS BAGUETTE

QUATREQUART CHOUQUETTE PARISBREST i PARISBREST , 0 CLAFOUTIS BAGUETTE
QUATREQUART CHOUQUETTE PARISBREST compte PARISBREST , 0 CLAFOUTIS BAGUETTE

ICECREAM PARISBREST boucle PARISBREST
MUFFIN
    QUATREQUART CHOUQUETTE PARISBREST courant PARISBREST , GAUFFREDEBRUXELLES CHOUQUETTE MADELEINE CHOUQUETTE PARIS

    SABLE CHAUSSONAUXPOMMES CHOUQUETTE TIRAMISU CHOUQUETTE MADELEINE CHOUQUETTE PARISBREST courant PARISBREST CLAFO
    FRAMBOISIER MUFFIN
        SABLE TIRAMISU CHOUQUETTE MADELEINE CHOUQUETTE PARISBREST courant PARISBREST CLAFOUTIS , MADELEINE CHOUQUET
        FRAMBOISIER MUFFIN
            QUATREQUART CHOUQUETTE PARISBREST compte PARISBREST , CANELE CHOUQUETTE MADELEINE CHOUQUETTE PARISBREST
        COOKIES BAGUETTE

        QUATREQUART CHOUQUETTE PARISBREST i PARISBREST , CANELE CHOUQUETTE MADELEINE CHOUQUETTE PARISBREST i PARISB
        PAINVIENNOIS PARISBREST boucle PARISBREST BAGUETTE
    COOKIES BAGUETTE
COOKIES BAGUETTE

PAINVIENNOIS PARISBREST boucle PARISBREST BAGUETTE

PAINAUCHOCOLAT CHOUQUETTE PARISBREST Voici le nombre de fois que %d apparait dans la phrase : %d PARISBREST , MADEL

PAINAUCHOCOLAT CHOUQUETTE PARISBREST Parmi %d caractère(s) PARISBREST , MADELEINE CHOUQUETTE PARISBREST i PARISBRES
```

Explication du fonctionnement

Le programme suit ces étapes :

1. Il lit la phrase avec `ECLAIR (0)` et lui ajoute le caractère `#` pour marquer sa fin.
2. Il lit le caractère à compter et initialise deux variables à zéro : `i` pour la longueur parcourue et `compte` pour le nombre d'occurrences.
3. La boucle `ICECREAM` / `PAINVIENNOIS` examine les caractères un par un avec `GAUFFREDEBRUXELLES`.
4. Tant que le caractère courant n'est pas `#`, le programme compare celui-ci avec le caractère recherché. En cas d'égalité, il incrémente `compte`, puis incrémente `i`.
5. À la fin de la phrase, il affiche le nombre d'occurrences et le nombre total de caractères.

Attention à deux limites : cette version suppose que la phrase ne contient pas `#`, car ce caractère sert de marqueur de fin. Elle est également destinée aux caractères ASCII, comme ceux des tests fournis ; les caractères accentués nécessiteraient une gestion adaptée de l'encodage.

Vérification sur les cas de test

Pour le premier test, avec `Voici une phrase de test` et le caractère `e`, les résultats attendus sont :

```
Voici le nombre de fois que e apparait dans la phrase : 4
Parmi 24 caractère(s)
```

Pour le deuxième test, avec `Une autre phrase de test` et le caractère `u` :

```
Voici le nombre de fois que u apparait dans la phrase : 1
Parmi 24 caractère(s)
```

La mise en forme diffère légèrement de celle de l'énoncé, ce qui est autorisé. Je n'ai pas pu exécuter l'interpréteur ici : ces résultats correspondent à la logique prévue du programme, et la syntaxe proposée s'appuie sur les exemples documentés.

Pour lancer le fichier avec ton interpréteur installé :

```
baguette_sharp --input p.baguette
```

You said:

baguette_sharp --input p8.baguette
Voici une phrase de test
e
Error: at line -1 while evaluating MADELEINE => Exception : The MADELEINE function does not exists do you mean MADELEINE ?
Error: at line -1 while evaluating MADELEINE => Exception : The MADELEINE function does not exists do you mean MADELEINE ?
Voici le nombre de fois que e apparait dans la phrase : 0
Parmi 1 caractère(s)

Worked for 2m 3s

ChatGPT said:

Tu as raison, ma première version contenait des erreurs de syntaxe et de gestion de la boucle. Désolé. `MADELEINE` est bien une fonction documentée de Baguette# qui permet d'accéder à une variable ; elle est également utilisée dans l'exemple officiel `turing.baguette` .

Esolang +1

Les deux problèmes à corriger sont notamment les parenthèses supplémentaires dans les opérations `CANELE` et le contrôle de la boucle, qui n'était pas correct.

Version corrigée de `p8.baguette`

Remplace le contenu du fichier par celui-ci :

```
QUATREQUART CHOUQUETTE PARISBREST phrase PARISBREST , GAUFFREDELIEGE CHOUQUETTE ECLAIR CHOUQUETTE 0 CLAFOUTIS , PAR
QUATREQUART CHOUQUETTE PARISBREST cible PARISBREST , ECLAIR CHOUQUETTE 0 CLAFOUTIS CLAFOUTIS BAGUETTE
QUATREQUART CHOUQUETTE PARISBREST i PARISBREST , 0 CLAFOUTIS BAGUETTE
QUATREQUART CHOUQUETTE PARISBREST compte PARISBREST , 0 CLAFOUTIS BAGUETTE

PAINVIENNOIS PARISBREST boucle PARISBREST BAGUETTE

ICECREAM PARISBREST boucle PARISBREST
MUFFIN
    QUATREQUART CHOUQUETTE PARISBREST courant PARISBREST , GAUFFREDEBRUXELLES CHOUQUETTE MADELEINE CHOUQUETTE PARIS

    SABLE CHAUSSONAUXPOMMES CHOUQUETTE TIRAMISU CHOUQUETTE MADELEINE CHOUQUETTE PARISBREST courant PARISBREST CLAFO
    FRAMBOISIER MUFFIN
        SABLE TIRAMISU CHOUQUETTE MADELEINE CHOUQUETTE PARISBREST courant PARISBREST CLAFOUTIS , MADELEINE CHOUQUET
        FRAMBOISIER MUFFIN
            QUATREQUART CHOUQUETTE PARISBREST compte PARISBREST , CANELE CHOUQUETTE MADELEINE CHOUQUETTE PARISBREST
        COOKIES BAGUETTE

        QUATREQUART CHOUQUETTE PARISBREST i PARISBREST , CANELE CHOUQUETTE MADELEINE CHOUQUETTE PARISBREST i PARISB
        PAINVIENNOIS PARISBREST boucle PARISBREST BAGUETTE
    COOKIES BAGUETTE
COOKIES BAGUETTE

PAINAUCHOCOLAT CHOUQUETTE PARISBREST Voici le nombre de fois que %d apparait dans la phrase : %d PARISBREST , MADEL

PAINAUCHOCOLAT CHOUQUETTE PARISBREST Parmi %d caractere(s) PARISBREST , MADELEINE CHOUQUETTE PARISBREST i PARISBRES
```

Attention : cette version est corrigée sur le plan de la structure et des parenthèses, mais je ne peux pas garantir son exécution sans disposer de l'interpréteur. Le programme utilise `#` comme marqueur de fin : il ne faut donc pas saisir ce caractère dans la phrase.

Enregistre le fichier, puis relance :

```
baguette_sharp --input p8.baguette
```

Pour le premier test, le résultat attendu reste **4 occurrences de e parmi 24 caractères**.

Si une nouvelle erreur apparaît, transmets-moi le message complet de l'interpréteur : il permettra de cibler précisément la syntaxe encore à ajuster.